Report for assignment 4

Report for assignment 4 1
Project 2
Onboarding experience 2
Architectural Overview and System purpose (P+) 3
Effort spent 4
Emma 4
Sumeya 5
Andy 6
Martin 7
Yiqin 8
Issue by Emma, Sumeya and Yiqin 11
Overview of issue and work done 11
Requirements 12
Existing relevant tests 13
New and/or updated tests (P+) 13
Code changes 14
Patch (P+) 14
Test results 15
UML class diagram and its description 15
Key changes/classes affected 15
Plan for completion 16
Issue by Martin 16
Existing relevant tests 18
Test results 18
Git Diff 18
UML 21
Status 22
Issue by Andy 22
Existing relevant tests 23
Test results 24
Git diff 24
--- a/tests/qt/test_field_widgets.py 29
UML 29
Status 30
Overall experience 31
Lessons learned 31

Team according to Essence standard.....31
Critical argumentation (Semat Analysis P+)..... 32

Project

Name: TagStudio

URL: https://github.com/TagStudioDev/TagStudio

TagStudio is an application used to organize files and photos by using a tag-based system.

Onboarding experience

Assignment 3 was done with the project ‘Apache Common Langs’, which worked well. However, the group decided to move on with another project, and came across ‘Sir Lancebot’. As this project seemed like a more ‘fun’ project, the group decided to proceed with onboarding to see if it was suitable or not. Everything seemed to be working out until it occurred that there were no pre existing unit tests, so the project was not suitable for this assignment, and a new project had to be found. Unfortunately, the lack of tests was noticed after setting up the project, hence a good lesson to verify that earlier in the onboarding if it is a criteria.

The next project that was decided to look into was ‘TagStudio’, where the existence of unit tests was verified straight away. The project has been verified to have more than 10K LOC with the command ‘cloc .’, which resulted in about 23 000 lines of Python code. Additionally, the project has been verified to not be a loose collection of algorithms.

The documentation was good for Apache Common Langs as it made it easy to build the project and run all tests when all required tools were listed. TagStudio had a lot of information in the README which at first glance seemed like a good sign that the onboarding and setup would be quick and smooth. However, there were some smaller bumps along the road where the getting started documentation was spread out in several places, which made it a bit hard to navigate at first. Secondly, there were some issues when running the unit tests directly after installing the developer environment and no real guides on what it could be. Turned out to be conflicts with Mac that with some googling could be solved.

Ultimately, TagStudio fulfilled the requirements and passed all tests which made the issue of a new feature fitting for this assignment.

Architectural Overview and System purpose (P+)

Tagstudio is an open-source multi-platform desktop application that is designed for managing large libraries of files through an advanced metadata-driven approach. The core purpose of the system is to provide users with a database-first view of their files. With tagstudio a user can discover and filter files by text, numeric values with min/max constraints (slider) and ratings (partly implemented).

Tagstudio solves this by implementing a Metadata-Driven Architecture. Instead of the user browsing a static disk structure, the system treats files as unique Entries within a dynamic, attribute-linked architecture. In a normal computer a file is defined by the folder it sits in, if you move the file, you lose its context. TagStudio changes this by separating the file's location from its identity, where every file gets its own ID in a database. This means that all the information you add stays connected to the file even if you move it to a different folder.

The Field in this system is typed metadata attributes that may be attached to entries. Before our implementation of numeric fields, the available field types were text (in the form of line, box, and date time), boolean and tags. This meant that the only way to describe and search for an Entry was with some text or dates, but not with numerical values. A big motivation for implementing numerical values is to be able to search for files in ranges of numerical values, especially since the bigger or smaller or equal operators are already functional in TagStudios query operations. Therefore, adding the option of numerical values to assign and use in the searches are both natural and useful to extend ways users can organize even more.

The system follows an architecture that separate the data storage from the user interface in such way:

1. Storage layer: Tagstudio uses SQLAlchemy models to interface with SQLite databases. The core entity is the Entry, which represents a physical file on the disk, where each entry has a relationship with different field types. Before our implementation the schema was only optimized for string and boolean types. By using Mapped [float], we ensured that the database itself enforces type safety.

2. Library Layer: The library class handles all transactions ensuring that the entries and their associated fields are correctly mapped. When a user adds a numeric value in the UI this layer is responsible for validation of the data, rolling back the session and ensuring the transaction is atomic.

3. UI layer: The system uses a widget rendering system and creates a widget which the UI asks for when it needs to display metadata for a file. It's a desktop interface built with QT. When the UI encounters a FieldTypeEnum.NUMERIC, the field container dynamically instantiates a NumericWidget. The same principle goes with the other fields: slider and rating (partly implemented).

The architecture (along with the affected/relevant classes) can be seen illustrated as a class diagram in the section “UML class diagram and its description”.

The strategy used for the implementation of the numerical value feature is referred to as test-driven development. This means that after the requirements have been defined the first step to implementation is by creating all the tests that would ensure a working functionality, with each of them failing expectedly. The tests were written at the UI layer, ensuring the widgets renders as expected, and the database layer, ensuring that values are correctly stored and retrieved for the different types.

By introducing the numeric field types (e.g. numeric, slider, rating) we enabled quantitative metadata, which filled a gap in the architecture. Before our implementations the system was optimized for categorical data and descriptive data. A critical gap we filled was the lack of structured hierarchy for quantitative data. By applying test driven development we established an inheritance chain that allows for future extension to even more types of fields. This shift from categorical to quantitative metadata transforms Tagstudio into a more powerful tool.

Effort spent

Emma

1. **Plenary discussions/meetings**; Due to lack of communication/commitment from big parts of the group all plenary things that should have been discussions or meetings were done by Emma. Did however receive input quickly from Sumeya when asked for via chat.
   Time spent: roughly **2,5 h** including finding issues and performing onboarding

2. **Discussions within parts of the group**; Continuous discussion via chat to parts of the group, adding up to about **1 h**

3. **Reading documentation**; Roughly **1 h** including documentation for setting up the project that later on turned out to be unfitting, and the documentation of the project that was finally chosen.

4. **Configuration and setup**; About **1,5 h** due to, as mentioned earlier, set up one project first but then moved on from it, which took around 1 hour due to many steps needed to take. Then the final project setup took about 30 minutes due to issues with homebrew python not being compatible, but also not being documented that it was not. Other dependencies were installed through pip without issues.

5. **Analyzing code/output**; Analyzing code has of course been consistently done, even at the onboarding stage when looking for projects. However, that is hard to put a time

stamp on, so I'll let that count in with the other relevant sections. What can be clearly said is that I spent around **1,5 h** analyzing the architecture/class connections and the existing tests.

6. **Writing documentation**; Writing documentation has consisted mainly of this report where I have written the onboarding experience, issue overview, requirements, lessons learned, SEMAT, patch, test results, plan for completion, and created the UML diagram. All sections relating to P+ have been written by me and Sumeya in collaboration. In total, this took about **10 h**. Writing this time log not included as it was *under* 30 min.

7. **Writing code**; I wrote UI tests for REQ-00x which took about 2 h (excluding REQ-00d). I also wrote UI and logic tests for all REQ-02[letter] which took about 2 h. Finally, I implemented the functionality of the slider field fulfilling the requirements REQ-00c and all of the REQ-02[letter] which took about 7 h due to a lot of debugging and errors encountered. Total time for writing code adds up to around **11 h**.

8. **Running code**; After the setup with the installations were complete, no more runs of the code took more than a couple minutes each. Hence running code has likely been less than 15 minutes total, and not worth including in total.

Total: Around **28,5 h**

Sumeya

1. **Plenary discussions/meetings;** Emma being the group leader she took initiative to get started. I did inform early on that working on the project early in the week would be difficult due to other assignments, but did contribute quickly via chat when Emma asked for input. After that there were very little discussions amongst the group, while me and Emma communicated throughout the whole time.

2. **Discussions within parts of the group;** Continuous discussion via chat to parts of the group, adding up to about 1 h.

3. **Reading documentation;** Studied TagStudios documentation, time spent: 1 h.

4. **Configuration and setup;**
   Since the setup exceeded 30 minutes, here is the enumeration of dependencies:
   - Python 3.12: This was needed since the project does not support older versions which I had.
   - Homebrew pathing: after installing Python 3.12 the terminal returned "command not found error". This was due to missing symlinks in the direction, which was

fixed by forcing homebrew to overwrite and repair the links. This took about 30 min.

- Test: when running “”pytest” the process terminated Fatal python error:aborted ". By analyzing the trace I could identify that it was a library that initialized GUY components. to just run the logic tests i change the command to pytest tests/macros.

In total this part took approximately 1 hour, a lot of time was spent just waiting for the terminal to execute tasks.

5. **Analyzing code/output;** Analyzing code/ has been done throughout the entire time, time spent on this is hard to say.
6. **Writing documentation;** Writing in the documentation took about 3 h. All sections relating to P+ have been written by me and Emma in collaboration.

7. **Writing code;** logic/Backend: i implemented the logic tests in test_numeric_logic.py for the backend. I also implemented the functionality for this and made sure all tests was passed. UI integration: I took over the implementation of the UI components based on emmas tests in test_field_widgets.py and implemented the functionality. During the process I refactored and modified the UI test code to resolve errors. This involved resolving malloc:corruption of free object errors on Macos, implementing proper QT parent-child ownership etc. I ensured that all 5 UI tests passed. time spent at 13h.
8. **Running code?** Running the code resulted in many errors, hence why I spent 13 hours “writing code”. I spent roughly 5-6 hours specifically on execution errors. Once the environment was stabilized the actual execution of the test was very efficient, taking only a few seconds per run. summing up all the iterative validation runs during the development cycle the total time would be 30-45 minutes.

Total: Around **19 h**

Andy

1. Plenary discussions/meetings;
   Started late and asked if there was anything I could work on. But ended up just finding another issue on the upstream project. ~30 min

2. Discussions within parts of the group;
   Mainly skimmed through the group chats to get a general understanding of where the project status was. ~5 min

3. Reading documentation;
   The documentation was rather simple to get started, the general idea was just to read `CONTRIBUTING.md` to get an understanding of the workflow and `README.md` to see what the project was about. ~20 min

4. Configuration and setup;
   It took less than 30 min.

5. Analyzing code/output;
   Mainly just reading through library.py, joins.py, visitors.py and constants.py to understand how tag deletion and parent-child search resolution works. Also analyzed existing tests in test_library.py and test_search.py to understand the test patterns. ~1 h

6. Writing documentation;
   Filled in specific sections related to my scope: issue summary, scope, requirements, test results, effort log, etc. ~1 hour

7. Writing code;
   Wrote 3 test cases in `test_tag_deletion.py` and the fix in `library.py` (remove_tag() method). Debugging the test took a small bit of time since I messed up the argument order in add_parent_tag, wrong BrowsingState method. ~1:30 hours

8. Running code?
   Running baseline tests in main and my branch. ~5 min

Total: 4-5 hours

Martin

1. Plenary discussions/meetings;
   Took around 20 minutes to find and commit on an interesting issue in the list of the issues of the project.

2. Discussions within parts of the group;
   Short exchanges via chat regarding which part of the ignore functionality to implement and how it interacts with refresh logic. Roughly 10 minutes.

3. Reading documentation;
   Read through relevant sections of the README and CONTRIBUTING.md, as well as documentation related to the ignore system and library initialization. Approximately 1 hour.
4. Configuration and setup;
   This part took longer than expected due to Python version incompatibility. The project requires Python 3.12, while my system was running a newer version, 3.14. Initially, I attempted to adjust the project configuration (e.g., modifying the TOML settings) to avoid installing an additional Python version. After spending around an hour trying to resolve the compatibility issues this way, I decided to install the required Python version instead. So running the project and getting the tests to pass took approximately one hour.

5. Analyzing code/output;
   Analyzed the existing ignore implementation, refresh logic, and library initialization flow to determine where the default .ts_ignore file should be generated. Also reviewed how ignored entries are tracked and removed. Approximately 1 hour.

6. Writing documentation;
   Updated the report with description of the issue, requirements, code changes, and test additions. Approximately 45 minutes.

7. Writing code;
   This part took a bit longer than initially expected. My first idea was to implement automatic generation of a default .ts_ignore template, which would have addressed the first part of the issue in a relatively straightforward way. I spent some time locating the appropriate place in the initialization logic to implement this, but upon further inspection realized that this functionality had already been implemented. I therefore shifted focus to the refresh-related part of the issue instead. This involved modifying the refresh workflow to include an additional step where ignored entries are re-evaluated and updated accordingly. In total, this part took approximately 3 hours.

8. Running code?
   Ran the full test suite before and after implementing the change to ensure no regressions were introduced. Also ran targeted tests while developing. Approximately 30 minutes.

Total:

Approximately 6-8 hours in total.

Yiqin

1. Plenary discussions/meetings;

2. Discussions within parts of the group;

3. Reading documentation;

4. Configuration and setup;
   Setting up took 30 minutes, where I had to first downgrade my Python version from 3.13.9 to 3.12, due to an explicit version compatibility error (ERROR: Package 'tagstudio' requires a different Python: 3.13.9 not in '<3.13,>=3.12'). Next, to isolate the project dependencies from my global Conda (base) environment, I created a separate venv within the Assignment 4 folder. I did encounter an ObjectNotFound and script execution

error when trying to run .\venv\Scripts\activate in terminal, but I resolved it by bypassing the system's execution policies so the activation script could run.

Lastly, I successfully installed dependencies according to CONTRIBUTING.md and executed pip install -e ".[dev]" in terminal.

```
Successfully built TagStudio
Installing collected packages: webencodings, texttable, pydub, paginate, distlib, brotli, altgraph, watchdog, urllib3, ujson, typing-extensions,
 types-ujson, toml, tinycss2, structlog, six, shiboken6, send2trash, semver, ruff, rarfile, pyyaml, pywin32-ctypes, pyreadline3, pyppmd, pyinsta
ller-hooks-contrib, pygments, pycryptodomex, pycparser, pybcj, psutil, pluggy, platformdirs, pillow, pefile, pathspec, numpy, nodeenv, mypy-exte
nsions, mutagen, multivolumefile, mkdocs-material-extensions, mergedeep, markupsafe, markdown, iniconfig, inflate64, idna, identify, greenlet, f
uture, filelock, defusedxml, coverage, colorama, charset-normalizer, chardet, cfgv, certifi, bracex, backrefs, backports-zstd, babel, attrs, ann
otated-types, wcmatch, useful_types, typing-inspection, sqlalchemy, requests, rawpy, pyzstd, pyyaml-env-tag, python-discovery, python-dateutil,
```

5. Analyzing code/output;
   Before any new tests are added, we have to check for the integrity of our forked TagStudio repository. So I executed the full automated test suite using the pytest tests/ command, taking approximately 1 minute and 17 seconds to run through all collected items. 231 tests passed with 0 failures, with warnings being mainly due to the older syntax used by the developers.

```
---------------------------------------------------------- snapshot report summary ---------------------
1 snapshot passed.
================================================= 231 passed, 456 warnings in 77.27s (0:01:17) ==========
```

   Issue 1312: [Bug] Favorite Stars do not add/remove from entire selection of items

6. Writing documentation;
   30 min used to draft REQ-04/05 requirements, analysed how this rating system differs from a standard slider, especially the need for icon configuration and integer steps. Also brainstormed how I am going to implement the TDD (Test-Driven Development) red-green cycle, which took another 1 hour.

7. Writing code;

   Took 1 hour to created the initial "Red" test suite in tests/test_rating_logic.py. These tests were designed to fail initially because the RatingField class was not created yet, resulting in an ImportError.

   (2 hours) Then I developed the logic for RatingField to inherit from the SliderField and added the RatingField class to fields.py, the import was successful and the logic for integer rounding worked.
   Also implemented a custom setter to ensure the rating value is always an integer (no partial stars) and added a configuration attribute for the default icon to be star.

To verify, I ran pytest such that "ImportError" (due to class missing) becomes "Passed".

```
import RatingField; r = Rating
 {r.icon_type}'); print(f'REQ-
--- Verification Success ---
REQ-04 (Icon): heart
REQ-05 (Integer): 5
```

8. Running code?

Total: 4.5 hr

Issue by Emma, Sumeya and Yiqin

Overview of issue and work done.

Title: [Feature request] Numeric Field Types

URL: https://github.com/TagStudioDev/TagStudio/issues/1141

Summary: The issue is a new feature about implementing the new field type ‘numeric field’ to allow users to make search operations using numeric comparison rather than just searching for files that contain a certain text or a date time. The issue contains three parts

1. A simple numeric input field.
2. A slider with min, max and step values, which inherits from the numeric input.
3. A rating field to configure displayed icons, which inherits from the slider.

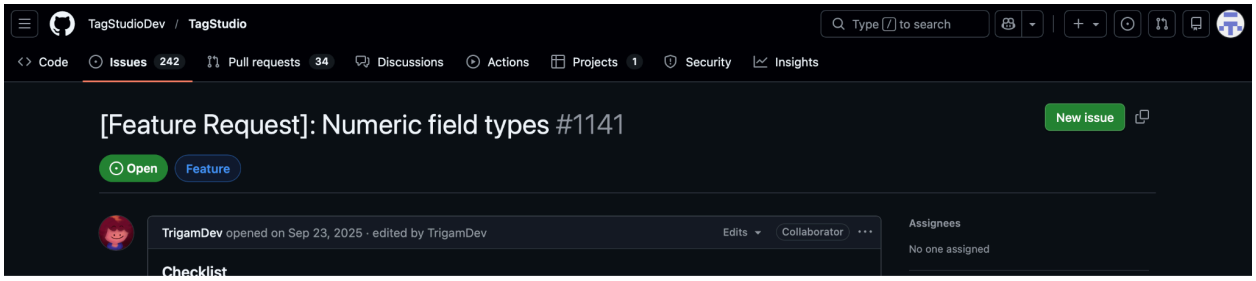
These new field types (and subtypes) will be stored in the database and displayed in the entry panel just like the other field types.

Scope:
By using the TDD approach, we ensured that the inheritance chain worked, such that

- NumericField: This provides the base value for all quantitative data, where it handles the core responsibility of storing a float value in the SQLAlchemy database.
- SliderField: Building upon the NumericField we implemented the SliderField. This introduces min, max and step attributes, ensuring that the user's interaction in the UI stays within a defined logical range.
- RatingField: This specializes in the visual representation and datarounding, where it adds icons and round about integers.

This hierarchical approach significantly reduces code duplication.

Note: The assignment said to assign an account to the issue, but this function is blocked for other users than moderators of the project. A screenshot is included to show that no one was assigned to the issue when the assignment started.



Requirements

ID	Description
REQ-00a	The text-based field types should render the text widget in entry panel
REQ-00b	The numeric field type should render the numeric widget in entry panel
REQ-00c	The slider field should render the slider widget in entry panel
REQ-00d	The rating field should render the rating widget in entry panel
REQ-01	The new feature should provide a simple numeric input field that stores numeric data
REQ-01a	The numeric input should be possible to insert into entry
REQ-01b	The numeric input should be able to be removed from the database
REQ-01c	The numeric input should be able to handle integers
REQ-01d	The numeric input should be able to handle floats
REQ-01e	The numeric value in entry should be able to be updated
REQ-01f	The numeric value should be able to be used in query operations
REQ-02	The new feature should provide a slider field type
REQ-02a	The slider field should have a configurable minimum value
REQ-02b	The slider field should have a configurable maximum value
REQ-02c	The slider field should have a configurable step value
REQ-02d	The slider field should handle values smaller than minimum by setting them as minimum
REQ-02e	The slider field should handle values greater than maximum by setting them as maximum
REQ-03	The slider field should inherit the same functionality as the numeric input field
REQ-04	A rating field should be added to configure displayed icons
REQ-04a	The rating field should allow a configurable number of icons (e.g., 5 stars vs 10 stars).
REQ-04b	The rating field should allow the user to select an icon representing a numerical value.
REQ-05	Rating field should have the same functionality as the slider field

REQ-05a	The rating field must store the selected value as an integer in the database.

Existing relevant tests

As the new feature is about adding more *field types*, the current tests for the text line field, datetime field and text box field, are relevant to analyze the structure and quality of.

The tests in 'test_library.py' contain meaningful assertions in the way that they do not just test asserting null, but rather test the behaviour that values in fields are updated correctly and that field counts are changing up and down as expected.

A test file that could seem relevant is 'test_field_containers.py' which does test containers that contain the field type input, but the focus is more on the visibility functionality of the containers itself. Therefore, tests that are currently missing, and even more important when adding more field types, is ensuring that the right field widget is rendered.

Also, 'test_library.py' does not have any prepared tests that validate input types or enforce numeric constraints. This is expected since numeric field type is an entirely new feature, and new tests need to be implemented in 'test_library.py' and a new test file 'test_field_widgets.py', targeting specifically numeric fields to ensure each requirement is being tested. To ensure high test quality, 'test_field_widgets.py' should also contain tests targeting the already existing field types as well, hence the requirement REQ-00 targets *different field types* and not just numeric field types.

New and/or updated tests (P+)

Initially, it was decided that test_library.py was supposed to be extended to test out logic of the new field types. However, it was later realised to be better managed to have separate logic test files for the different field types.

The tracing of tests have been completed for the numeric field and slider field, as those have been implemented by Sumeya and Emma who are aiming for P+.

File	Name of test	Requirement(s)
tests/qt/test_field_widgets.py	test_text_line_renders_text_widget	REQ-00a
tests/qt/test_field_widgets.py	test_text_box_renders_text_widget	REQ-00a
tests/qt/test_field_widgets.py	test_date_time_renders_text_widget	REQ-00a
tests/qt/test_field_widgets.py	test_numeric_renders_numeric_widget	REQ-00b, REQ-01
tests/qt/test_field_widgets.py	test_slider_renders_slider_widget	REQ-00c, REQ-02

tests/test_numeric_logic.py	test_add_numeric_field_to_entry	REQ-01a
tests/test_numeric_logic.py	test_numeric_field_stores_integer	REQ-01c
tests/test_numeric_logic.py	test_update_numeric_field	REQ-01e
tests/test_numeric_logic.py	test_remove_numeric_field	REQ-01b
tests/test_numeric_logic.py	test_numeric_comparison_query	REQ-01f
tests/test_numeric_logic.py	test_numeric_field_handles_float	REQ-01d
tests/test_slider_logic.py	test_slider_has_configurable_minimum	REQ-02a
tests/test_slider_logic.py	test_slider_has_configurable_maximum	REQ-02b
tests/test_slider_logic.py	test_slider_has_configurable_step	REQ-02c
tests/test_slider_logic.py	test_slider_handles_value_below_minimum	REQ-02d
tests/test_slider_logic.py	test_slider_handles_value_above_maximum	REQ-02e
tests/test_slider_logic.py	test_slider_inherits_numeric_functionality	REQ-03

Code changes

Patch (P+)

The following command in the branch **feat/numeric-field** will show the patch with changes by Emma and Sumeya (REQ-00, REQ-01, REQ-02), ensuring a clean patch for P+:

```
git diff main…feat/numeric-field > patch_temp.diff
```

The patch has also been uploaded in docs/patch_emma_sumeya.diff in the branch **feat/numeric-field**.

The patch is defined to be clean since:

- It does not include any new out-commented code.
- There are no unnecessary debug outputs committed.
- There are no unnecessary whitespace changes.

Also per the acceptance requirements specifically for TagStudio:

- The commits are atomic and conventional
- The code includes valid licenses
- The code has been formatted with ruff to ensure checks pass

Test results

The test logs relating to the numeric field and the slider field can be found in the branch **feat/numeric-field** in docs/testlogs/.

There are 4 logs:

- **test_log_before_tests.txt** where all tests were runned before any tests or implementation had been done.
- **test_log_REQ01_before_implementation** where all tests were runned after numeric field tests had been implemented, but no functionality had been implemented. The new tests failed as expected.
- **test_log_REQ02_before_implementation** where all tests were runned after slider field tests had been implemented, but no functionality had been implemented. The new tests failed as expected.
- **test_log_after_implementation** where all tests were runned after the functionality of the numeric field and the slider field had been implemented.

Note: As the tests (and functionality) for the rating field were implemented by Yiqin but no test logs are added here, we (Emma and Sumeya) can not be responsible for that they work due to lack of commitment and communication from Yiqin.

UML class diagram and its description

Key changes/classes affected

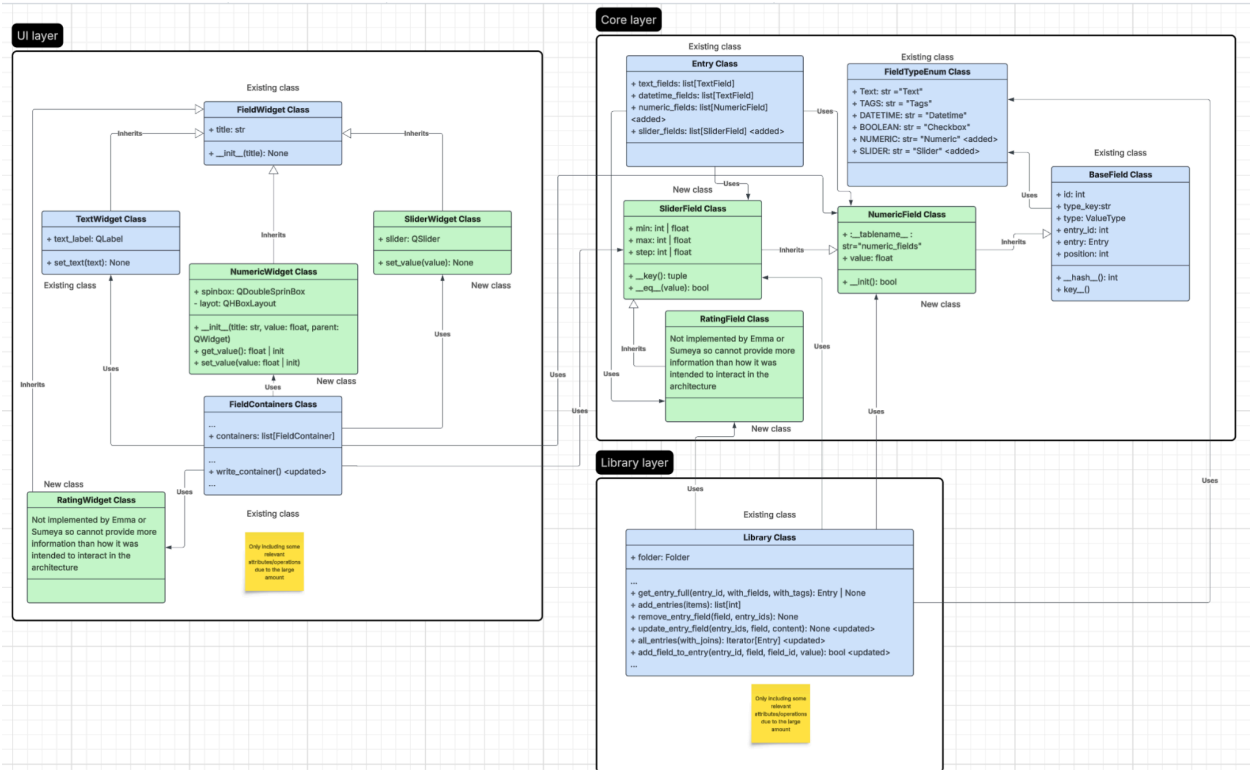
The UML class diagram below shows the new (green) classes and the existing (blue) classes, that either are important for how the new classes interact in the architecture and/or what has been updated in them in order for the new fields to work properly.

The affected layers are the UI layer, library layer, and core layer. The UI layer includes the new widget classes for each of the new fields that inherits from the already existing field widget class. Then the widgets are used by the field container in operations in order to display the widgets properly, which also uses the new field classes in the core layer. The field container class has been updated to handle the new fields.

The library layer contains several operations to handle values in the database and it mainly interacts with the core layer defining the fields and entries. Multiple operations in the library section have been updated to handle the new fields correctly in order to save, remove, update and retrieve values to and from the database with different formats.

The core layer includes the new field classes defining what they need to be able to handle and their format. They are intended to inherit as rating→slider→numeric→base ensuring

each field has the same functionality as their parent with specialised functionality extending, but as of now the rating has not been fully integrated with the other fields.



Plan for completion

All of the defined requirements for the numeric field and the slider field have been tested and implemented to ensure all tests pass. As earlier mentioned due to lack of communication the status of the rating field is uncertain. If we were to complete the feature fully we would need to examine the rating tests to ensure they cover everything that is needed and then make sure the implementation ensures all those tests pass. After that we would need to merge the rating field together with the numeric and slider field, which likely would need some refactoring to ensure inheritance works as intended. Estimated time to complete would probably be around 5-6 hours with some margins for debugging, if 2 people were to do it. If 5 people, as intended from the start, were to do it, it could likely be done in a couple effective hours.

Issue by Martin

Title: Improve ignore handling during refresh

URL: https://github.com/TagStudioDev/TagStudio/issues/1269

Summary:
The issue concerns improving the handling of ignored files within TagStudio's refresh workflow. When users define ignore rules (via the .ts_ignore file), files that match these patterns should not remain in the library after a refresh. The goal of this contribution was to ensure that the refresh process correctly re-evaluates ignored entries and updates the library accordingly.

Previously, ignored files could remain in the database even after new ignore rules were introduced. The implemented change ensures that the refresh pipeline includes a step where ignored entries are identified and removed without affecting the actual files on disk.

Scope (functionality and code affected):
The work primarily affected the refresh logic within the library core. The following components were analyzed and modified:

- RefreshTracker in core/library/refresh.py
- IgnoredRegistry in core/library/alchemy/registries/ignored_registry.py
- The ignore pattern handling in core/library/ignore.py
- The library initialization and refresh workflow

The refresh pipeline was adjusted so that ignored entries are re-evaluated after scanning and removed from the library when necessary. Care was taken to ensure that:

- Existing functionality was not broken
- Files were only removed from the database, not from disk
- All existing tests continued to pass

A new test was added to verify refresh behavior and ensure that new files are correctly detected and a new tested added to make sure files are not removed from the machine file system.

Note: As with the other issue, assigning an account to the issue was not possible since only project maintainers can assign contributors.

ID	Description
REQ-M1	During refresh, ignore rules defined in .ts_ignore must be re-evaluated.
REQ-M2	Entries matching ignore patterns must be removed from the database.
REQ-M3	Removing ignored entries must not delete the corresponding files from disk

Existing relevant tests

There are currently no tests verifying whether ignored files are actually ignored. However, there are some tests related to the refresh mechanism. These tests only ensure that when a new file is added, it is correctly integrated into the Tag Studio file system.

File	Name of test	Requirement
tests/macros/test_refresh_dir.py	test_refresh_tracker_adds_new_file	REQ-M1, REQ-M2
tests/macros/test_refresh_dir.py	test_removing_entry_does_not_delete_file	REQ-M3

Test results

Before I add the changes all 231 tests were passing.
Once I added 2 tests, all 234 tests are now passing, still 0 fail.

Git Diff

```
diff --git a/src/tagstudio/qt/ts_qt.py b/src/tagstudio/qt/ts_qt.py
index ab81b4e..33e071b 100644
--- a/src/tagstudio/qt/ts_qt.py
+++ b/src/tagstudio/qt/ts_qt.py
@@ -103,6 +103,10 @@ from tagstudio.qt.views.main_window import MainWindow
 from tagstudio.qt.views.panel_modal import PanelModal
 from tagstudio.qt.views.splash import SplashScreen

+from tagstudio.core.library.alchemy.registries.ignored_registry import IgnoredRegistry
```

```
+from tagstudio.core.library.ignore import Ignore
+from tagstudio.qt.utils.function_iterator import FunctionIterator
+
 BADGE_TAGS = {
     BadgeType.FAVORITE: TAG_FAVORITE,
     BadgeType.ARCHIVED: TAG_ARCHIVED,
@@ -1097,14 +1101,28 @@ class QtDriver(DriverMixin, QObject):
            )
        )
        r = CustomRunnable(iterator.run)
-       r.done.connect(
-           lambda: (
-               pw.hide(),
-               pw.deleteLater(),
-               # refresh the library only when new items are added
-               files_count and self.update_browsing_state(),  # type: ignore
-           )
-       )
+       def on_refresh_done():
+           pw.hide()
+           pw.deleteLater()
+
+           # Refresh ignore patterns
+           Ignore.get_patterns(self.lib.library_dir, include_global=True)
+
+           # Create registry instance
+           registry = IgnoredRegistry(self.lib)
+
+           # Run ignored detection
+           iterator = FunctionIterator(registry.refresh_ignored_entries)
+           iterator.run()
+
+           # Remove ignored entries
+           registry.remove_ignored_entries()
+
+           # Always update UI (safer)
+           self.update_browsing_state()
+
+       r.done.connect(on_refresh_done)
+
        QThreadPool.globalInstance().start(r)

    def new_file_macros_runnable(self, new_ids):
```

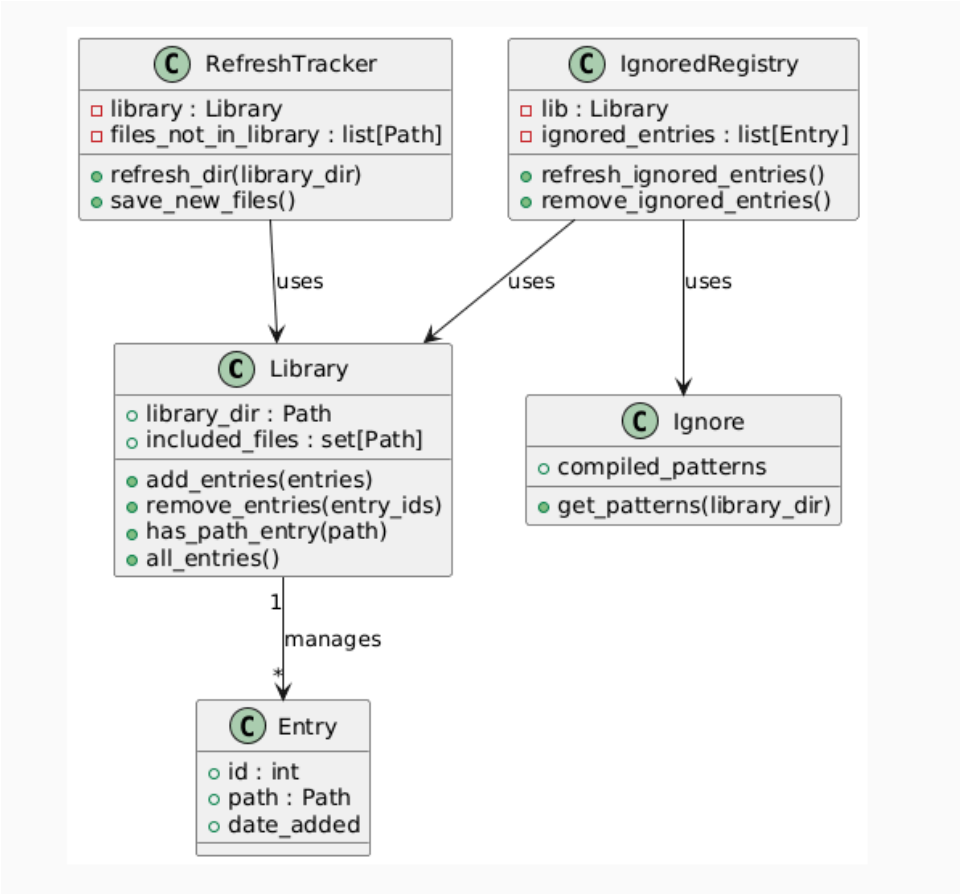
```
diff --git a/tests/macros/test_refresh_dir.py b/tests/macros/test_refresh_dir.py
index 4b550a2..f7c6165 100644
--- a/tests/macros/test_refresh_dir.py
+++ b/tests/macros/test_refresh_dir.py
@@ -11,6 +11,15 @@ from tagstudio.core.enums import LibraryPrefs
 from tagstudio.core.library.alchemy.library import Library
 from tagstudio.core.library.refresh import RefreshTracker
 from tagstudio.core.utils.types import unwrap
+from tagstudio.core.library.alchemy.registries.ignored_registry import IgnoredRegistry
+from tagstudio.core.library.ignore import Ignore
+
+from tagstudio.core.library.refresh import RefreshTracker
+from tagstudio.core.library.alchemy.registries.ignored_registry import IgnoredRegistry
+from tagstudio.core.library.ignore import Ignore
+from tagstudio.core.utils.types import unwrap
+from tagstudio.core.library.alchemy.library import Library
+

 CWD = Path(__file__).parent

@@ -49,3 +58,36 @@ def test_refresh_multi_byte_filenames(library: Library):
    assert Path("em–dash.txt") in registry.files_not_in_library
    assert Path("apostrophe’.txt") in registry.files_not_in_library
    assert Path("umlaute äöü.txt") in registry.files_not_in_library
+
+
+@pytest.mark.parametrize("library", [TemporaryDirectory()], indirect=True)
+def test_refresh_tracker_adds_new_file(library: Library):
+    library_dir = unwrap(library.library_dir)
+
+    # Create a file
+    file_path = library_dir / "example.txt"
+    file_path.touch()
+
+    tracker = RefreshTracker(library=library)
+    library.included_files.clear()
+
+    # Run refresh
+    list(tracker.refresh_dir(library_dir, force_internal_tools=True))
+
+    # The file should be detected as new
+    assert Path("example.txt") in tracker.files_not_in_library
+
```

```
+
+@pytest.mark.parametrize("library", [TemporaryDirectory()], indirect=True)
+def test_removing_entry_does_not_delete_file(library: Library):
+    library_dir = unwrap(library.library_dir)
+
+    file_path = library_dir / "file.txt"
+    file_path.touch()
+
+    tracker = RefreshTracker(library)
+    library.included_files.clear()
+    list(tracker.refresh_dir(library_dir, force_internal_tools=True))
+    list(tracker.save_new_files())
+
+    assert file_path.exists()
```

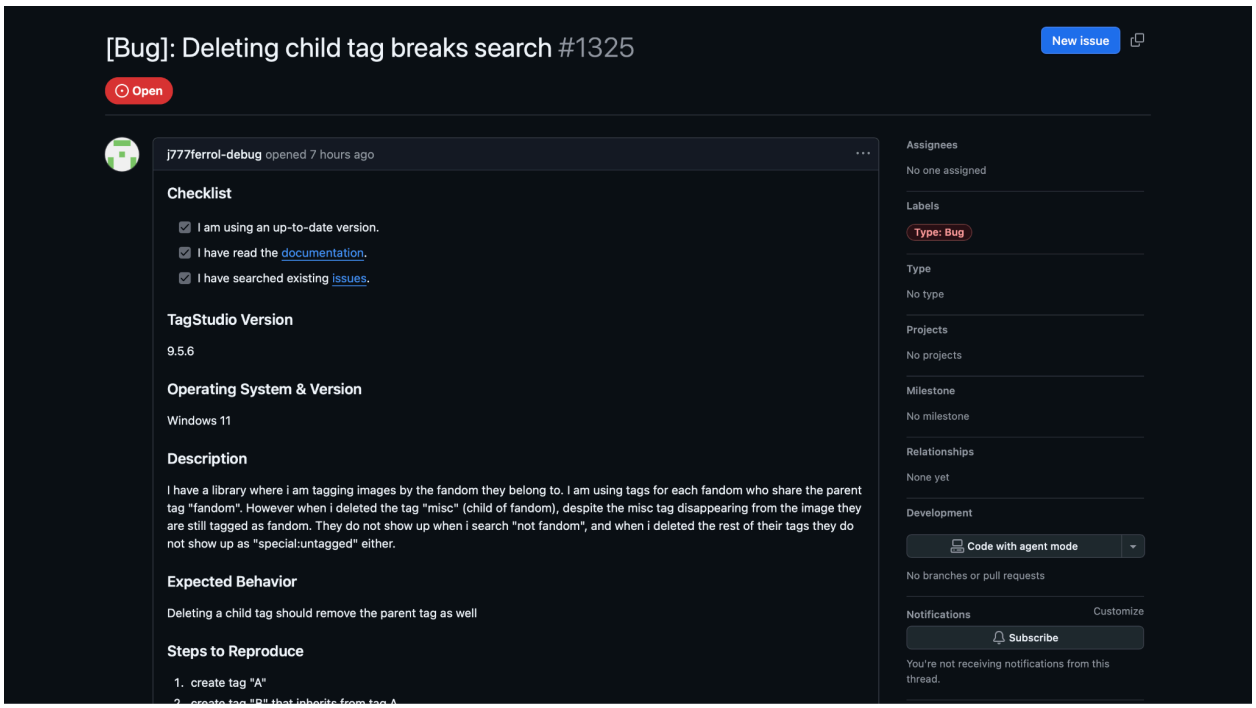
UML



Status

The feature has been implemented and all tests are passing. When the page is refreshed, the .ts_ignored file is re-evaluated: the corresponding files are removed from the Tag Studio file system, but they remain in the local machine file system. No pull request has been submitted yet.

Issue by Andy



Title: [Bug] Deleting tag breaks search

URL: https://github.com/TagStudioDev/TagStudio/issues/1325

Summary:
The fix I implemented migrates entries to the parent tag before the child tag is deleted.

The core issue was a bug tag related bug where deleting a child tag causes entries to disappear from parent tag searches. A simple example is where a tag “rock” is a child of “genre” and a file is tagged with “rock”, deleting “rock“ makes the file unsearchable under “genre” and it also doesn’t show up as untagged.
Scope (functionality and code affected):
The main bug was in the `remove_tag()` method in `core/library/alchemy/library.py`. The method simply deletes `TagEntry` rows (which link entries to tags) without explicitly checking if entries

need to be reassigned to parent tags. The fix therefore adds a migration step that firstly queries `TagParent` to find parent tags and inserts new `TagEntry` rows before deletion.

This scope therefore relates to the following files:

- library.py (`remove_tag()`)
- joins.py (`TagParent` and `TagEntry` table models)
- visitors.py (search query builder that resolves parent-child tag hierarchy)
- constants.py (recursive SQL query for resolving child tags)

ID	Title	Description
REQ-A01	Entries migrate to parent tag on child deletion	When a child tag is deleted, entries tagged only with that child should receive a TagEntry for the parent tag
REQ-A02	Parent tag search works after child deletion	After deleting a child tag, searching for the parent tag should still return entries that were previously found through the child
REQ-A03	Entries become untagged when all tags removed	When both child and parent tags are deleted, entries should appear in special:untagged searches
REQ-A04	Direct parent tags survive child deletion	If an entry has both parent and child tags directly applied, deleting the child should not affect the parent tag
REQ-A05	Existing functionality unaffected	All previously passing tests must continue to pass after the fix

Existing relevant tests

The existing test test_remove_tag in test_library.py only verifies that the tag count decreases after deletion. This does not check what happens to entries that were tagged with the deleted tag, or whether search results are affected. test_parent_tags in test_search.py tests that searching for a parent tag finds entries tagged with child tags, but it never deletes a tag. No existing test combines tag deletion with search verification, which is the core of this bug.

File	Name of test	Requirement(s)
tests/test_tag_deletion.py	test_delete_child_tag_preserves_parent_search	REQ-A01, REQ-A02
tests/test_tag_deletion.py	test_delete_child_tag_entry_becomes_untagged	REQ-A03
tests/test_tag_deletion.py	test_delete_child_tag_keeps_direct_parent_tag	REQ-A04
(full test suite — 234 passed)	all existing tests	REQ-A05

Test results

Before changes (main branch): 231 tests passed, 0 failed.
After tests written, before fix (RED): 1 failed (test_delete_child_tag_preserves_parent_search), 2 passed.
After fix (GREEN): 234 tests passed, 0 failed.

Git diff

```diff
@@ -1173,6 +1173,38 @@ class Library:
    def remove_tag(self, tag_id: int) -> bool:
        with Session(self.engine, expire_on_commit=False) as session:
            try:
+               # Migrate entries from this tag to its parent tags before deletion
+               # so entries remain searchable under the parent hierarchy.
+               parent_ids = [
+                   row.parent_id
+                   for row in session.execute(
+                       select(TagParent.parent_id).where(TagParent.child_id == tag_id)
+                   ).all()
+               ]
+               if parent_ids:
+                   entry_ids = [
+                       row.entry_id
+                       for row in session.execute(
+                           select(TagEntry.entry_id).where(TagEntry.tag_id == tag_id)
+                       ).all()
+                   ]
```

```
+                for parent_id in parent_ids:
+                    for entry_id in entry_ids:
+                        already_exists = session.execute(
+                            select(TagEntry).where(
+                                and_(
+                                    TagEntry.tag_id == parent_id,
+                                    TagEntry.entry_id == entry_id,
+                                )
+                            )
+                        ).first()
+                        if not already_exists:
+                            session.execute(
+                                TagEntry.__table__.insert().values(
+                                    tag_id=parent_id, entry_id=entry_id
+                                )
+                            )
+
            session.execute(delete(TagAlias).where(TagAlias.tag_id == tag_id))
            session.execute(delete(TagEntry).where(TagEntry.tag_id == tag_id))
            session.execute(
diff --git a/tests/test_tag_deletion.py b/tests/test_tag_deletion.py
new file mode 100644
index 0000000..b230837
--- /dev/null
+++ b/tests/test_tag_deletion.py
@@ -0,0 +1,110 @@
+# Copyright (C) 2025
+# Licensed under the GPL-3.0 License.
+# Created for TagStudio: https://github.com/CyanVoxel/TagStudio
+
+from collections.abc import Callable
+from pathlib import Path
+
+from tagstudio.core.library.alchemy.enums import BrowsingState
+from tagstudio.core.library.alchemy.library import Library
+from tagstudio.core.library.alchemy.models import Entry, Tag
+from tagstudio.core.utils.types import unwrap
+
+
+def test_delete_child_tag_preserves_parent_search(
+    library: Library, generate_tag: Callable[..., Tag]
+):
+    """When a child tag is deleted, entries tagged only with the child
```

```
+    should still appear in searches for the parent tag."""
+    # create parent tag A and child tag B
+    parent = unwrap(library.add_tag(generate_tag("genre", id=3000)))
+    child = unwrap(library.add_tag(generate_tag("rock", id=3001)))
+    library.add_parent_tag(parent.id, child.id)
+
+    # create an entry tagged with only the child tag
+    entry = Entry(
+        path=Path("song.mp3"),
+        folder=unwrap(library.folder),
+        fields=library.default_fields,
+    )
+    entry_id = library.add_entries([entry])[0]
+    library.add_tags_to_entries(entry_id, child.id)
+
+    # searching for the parent should find the entry (via child hierarchy)
+    results = library.search_library(
+        BrowsingState.from_search_query("genre"),
+        page_size=500,
+    )
+    assert results.total_count >= 1
+
+    # delete the child tag
+    library.remove_tag(child.id)
+
+    # searching for the parent should still find the entry
+    results = library.search_library(
+        BrowsingState.from_search_query("genre"),
+        page_size=500,
+    )
+    assert results.total_count >= 1
+
+
+def test_delete_child_tag_entry_becomes_untagged(
+    library: Library, generate_tag: Callable[..., Tag]
+):
+    """When a child tag is deleted and the entry has no other tags,
+    it should appear in special:untagged searches."""
+    parent = unwrap(library.add_tag(generate_tag("category", id=3100)))
+    child = unwrap(library.add_tag(generate_tag("subcategory", id=3101)))
+    library.add_parent_tag(parent.id, child.id)
+
+    entry = Entry(
```

```
+        path=Path("lonely_file.txt"),
+        folder=unwrap(library.folder),
+        fields=library.default_fields,
+    )
+    entry_id = library.add_entries([entry])[0]
+    library.add_tags_to_entries(entry_id, child.id)
+
+    # entry should not be untagged right now
+    results = library.search_library(
+        BrowsingState.from_search_query("special:untagged"),
+        page_size=500,
+    )
+    untagged_before = results.total_count
+
+    # delete both child and parent tags
+    library.remove_tag(child.id)
+    library.remove_tag(parent.id)
+
+    # entry should now appear as untagged
+    results = library.search_library(
+        BrowsingState.from_search_query("special:untagged"),
+        page_size=500,
+    )
+    assert results.total_count > untagged_before
+
+
+def test_delete_child_tag_keeps_direct_parent_tag(
+    library: Library, generate_tag: Callable[..., Tag]
+):
+    """When an entry has both parent and child tags directly applied,
+    deleting the child should leave the parent tag intact."""
+    parent = unwrap(library.add_tag(generate_tag("animal", id=3200)))
+    child = unwrap(library.add_tag(generate_tag("dog", id=3201)))
+    library.add_parent_tag(parent.id, child.id)
+
+    entry = Entry(
+        path=Path("pet_photo.jpg"),
+        folder=unwrap(library.folder),
+        fields=library.default_fields,
+    )
+    entry_id = library.add_entries([entry])[0]
+    library.add_tags_to_entries(entry_id, [child.id, parent.id])
+
```

```
+    # delete the child tag
+    library.remove_tag(child.id)
+
+    # parent tag should still be directly on the entry
+    refreshed = unwrap(library.get_entry_full(entry_id))
+    tag_ids = [t.id for t in refreshed.tags]
+    assert parent.id in tag_ids
```

—----------

--- a/src/tagstudio/core/library/alchemy/enums.py

```
class FieldTypeEnum(enum.Enum):
     TAGS = "Tags"
     DATETIME = "Datetime"
     BOOLEAN = "Checkbox"
+     NUMERIC = "Numeric"

+++ b/src/tagstudio/core/library/alchemy/enums.py
class FieldTypeEnum(enum.Enum):
     TAGS = "Tags"
     DATETIME = "Datetime"
     BOOLEAN = "Checkbox"
+     NUMERIC = "Numeric"
```

—----------

--- a/src/tagstudio/core/library/alchemy/fields.py

```
+++ b/src/tagstudio/core/library/alchemy/fields.py
+class NumericField(BaseField):
+    """SQLAlchemy model for storing numeric metadata."""
+    __tablename__ = "numeric_fields"
+    value: Mapped[float] = mapped_column(nullable=False)
```

—----------

--- a/src/tagstudio/qt/mixed/field_containers.py

```
+++ b/src/tagstudio/qt/mixed/field_containers.py
class FieldContainer(QWidget):
-        if field.value_type == "TEXT":
+        if field.value_type == "NUMERIC":
+            widget = NumericWidget(field.name, field.value, parent=self)
+        elif field.value_type == "TEXT":
             widget = TextWidget(field.name, field.value, parent=self)
```

—----------

--- a/tests/qt/test_field_widgets.py

```
+++ b/tests/qt/test_field_widgets.py
-def test_numeric_renders_numeric_widget(qt_driver, library, entry_full):
-    panel = PreviewPanel(library, qt_driver)
```

```
+def test_numeric_renders_numeric_widget(qt_driver, library, entry_full, qtbot):
+    panel = PreviewPanel(library, qt_driver)
+    qtbot.addWidget(panel)  # Ensures the widget lifecycle is managed by pytest-qt
```

——----------

--- a/src/tagstudio/core/library/alchemy/library.py

```
+    def get_entry_full(self, entry_id: int):
+        # Logic updated to include numeric_fields in the joined load
+        # to ensure they are returned to the UI.
```

——----------

--- a/src/tagstudio/qt/mixed/field_containers.py

```
class FieldContainer(QWidget):
-        if field.value_type == "TEXT":
+        if field.value_type == "NUMERIC":
+            widget = NumericWidget(field.name, field.value, parent=self)
+        elif field.value_type == "TEXT":
            widget = TextWidget(field.name, field.value, parent=self)
```

——----------

--- a/tests/test_numeric_logic.py

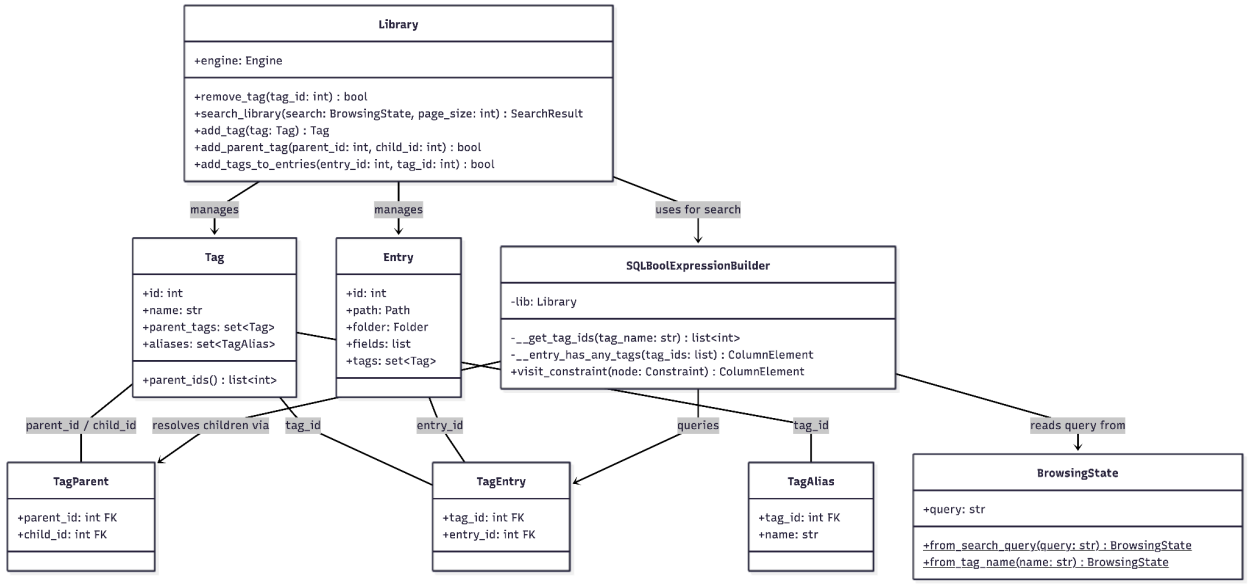
```
+def test_numeric_field_stores_numeric_data(library: Library):
+    """REQ-01: Verifies that numeric data can be persisted and retrieved."""
+    entry = Entry(path=Path("test.txt"), fields=[NumericField(type_key="QTY", value=100)])
+    entry_id = library.add_entries([entry])[0]
+    refreshed = library.get_entry_full(entry_id)
+    assert refreshed.numeric_fields[0].value == 100
```

——----------

--- a/tests/qt/test_field_widgets.py

```
-def test_numeric_renders_numeric_widget(qt_driver, library, entry_full):
-    panel = PreviewPanel(library, qt_driver)
+def test_numeric_renders_numeric_widget(qt_driver, library, entry_full, qtbot):
+    panel = PreviewPanel(library, qt_driver)
+    qtbot.addWidget(panel)  # Fixes "malloc: Corruption of free object" on macOS
```

UML



Status

The issue is fully resolved. The fix has been implemented, all tests pass (234/234), and submitted PR request to fork main.

Overall experience

Lessons learned

It has been very educational to work in the opposite order than what we usually do. Sometimes you can get stuck or overwhelmed when implementing a new feature and not knowing where to start, and by instead starting with writing concrete tests of “what should work”, gave us a fresh perspective and some of us even got started quicker than usual.

Another take-away from this project is that properly listing the requirements was useful to make a detailed and structured view of what needs to be tested, and therefore, implemented. A clear requirement list combined with a good understanding of the assignment and project architecture makes it easier to work parallel to one another without accidentally making conflicting work.

Finally this project has been another good way to learn more about bigger projects and how to further develop on things that you did not start, which is a valuable lesson for after graduation when you will often encounter projects that you need to jump into and know how to adapt to.

Team according to Essence standard

Unfortunately, the team is still at the *seeded* state, even after four assignments. By evaluating the Essence standard checklist in the seeded state we do fulfill the requirements as they are focused on everyone *understanding* what should be done and how. The foundation is set both by the group in earlier assignments on where and how to perform work, and by the assignment itself on how much time should be spent.

When looking at the next state, which is *formed*, many of the requirements are fulfilled:

- Enough team members are in the group to be able to finish the work
- The roles of the team are clear
- The team have met
- The team understands the responsibilities
- The communication method is defined

However, while the team members understand what is expected of them, not everyone puts in the effort to actually know how to perform their work, for example by not reading the assignment carefully before starting their work.

The group has been closer to higher states in earlier assignments, and realistically should be able to reach the next state if everyone starts taking initiative to accept work, just not coding, but everything related to software engineering practices. The group also needs to start to communicate earlier and more consistently, in order to avoid stress near the deadlines. When evaluating with essence it is important to know that states can change and to truly be in a state means that it should be consistent.

The conclusion is, that in order for the group to reach the next state is not to define more rules or methods, but rather that each member engages more and takes their fair share of responsibility, in order to use the defined rules and methods. By improving this behaviour, there would be a good and competent team working together, and not just individuals who happen to work on the same project.

Critical argumentation (Semat Analysis P+)

While the team had some difficulties with commitment and communication, the work itself can still be evaluated with relation to alphas and software engineering in the real world.

When evaluating the project it becomes clear that our work involves a balance between advancing the software system and managing the practical obstacles within the development environment. The implementation of numericField moved the system toward a much more mature state progressing it from *Architecture Selected* to *Stable*.

Regarding the requirements alpha we successfully fulfilled the functional goals for numeric data handling. However the process raised some concerns about the non-functional requirements, since we faced some issues for example with environment-specific crashes on macOS, where the tests kept running against the installed package instead of the source code, resulting in bus errors or interruptions. This indicates that the requirements for cross-platform stability were not sufficiently documented in the original project, and the setup documentation could be more clear about editable installs. From a software engineering perspective a requirement cannot be considered truly fulfilled until it operates across all intended environments. In our case reaching a *stable state* required modification such as making sure that the mapping between numeric field and slider field correctly inherits according to SQLAlchemy as well as solving conflicts appearing in the entry model relating to the relationships.

Regarding the work alpha, the implementation progressed from *Initiated* to *Under control*, mainly because we committed to a Test-Driven Development (TDD) approach, meaning we implemented tests that failed before we implemented the functionality. By letting the backend tests drive the logic, we ensured that the work progressed in a traceable manner. Finally our contributions mean that more people can use the tool more efficiently by searching for numbers and using sliderfield.

Even though our team is still in an early stage, evaluated according with Essence standard, and learning how to work together, the technical parts of the project have taken a big step towards becoming a stable product that is ready to be used.